
Pacote do R para coleta de dados em campo por dispositivo móvel utilizando o ODK Collect

CONTEXTO

O procedimento de coleta de dados em campo tem fontes de incertezas e erros envolvidas na aquisição da informação. Entre as fontes de erros mais importantes nos inventários florestais estão: anotação do dado em campo e transcrição da informação para o formato digital. Esses erros são custosos para serem resolvidos, muitas vezes demandando o retorno à campo, o que nem sempre é possível. Evitar fontes de erros no procedimento científico é essencial para não comprometer as conclusões dos trabalhos científicos. O pacote **odkTreeCensus** tem como objetivos minimizar a entrada de dados com discrepâncias durante a aquisição da informação e eliminar a fase de transcrição de dados anotados em papel. O pacote auxilia na construção e gerenciamento de formulários de entrada de dados para o aplicativo ODK-Collect, criando os arquivos necessários e gerenciando a entrada e saída de dados.

Autor: Alexandre Adalardo de OLIVEIRA
Departamento de Ecologia - IBUSP



Laboratório de Ecologia de Florestas Tropicais

Índice

Instalando pacote odkTreeCensus	2
Criar arquivos de dados acessórios	2
Códigos das subparcelas	2
Dados das árvores do último censo e códigos de subparcelas	3
Equipe	3
Lista de Espécies	3
Dados de tags por subquadrat	4
Mapas e Grids svg	4
Os rótulos para os svg	4
gridid.csv	4
Seleção de subquadrat	5
Mapas e Grids para seleção e mapeamento de árvores	5
ODK Collect	5
Dispositivo android	5
Android Debug Bridge	6
Preparando o dispositivo móvel	6
Criando um projeto ODK Collect	6
Conectando o Dispositivo Android	6
Configurando o USB Padrão	6
Criar projeto no ODK Collect	8
Atualizar o formulário xml	8
Copia o caminho do formulário XML	8
Subindo o formulário para o dispositivo móvel android	9

Instalando pacote odkTreeCensus

```
## verifica se tem o pacote remotes e caso nao tenha instala
if(!"remotes" %in% rownames(installed.packages()))
{
  install.package("remotes")
}
## Verifica se o odkTreeCensus está carregado na sessão
if("package:odkTreeCensus" %in% search())
{
  detach("package:odkTreeCensus", unload = TRUE)
}
## verifica se tem o odkTreeCensus e desinstala
if("odkTreeCensus" %in% rownames(installed.packages()))
{
  remove.packages("odkTreeCensus")
}
## instala a versao recente do odkTreeCensus

remotes::install_github("adalardo/odkTreeCensus")
library("odkTreeCensus")
```

Criar arquivos de dados acessórios

Todos os arquivos de dados que acompanham o formulário no ODK Collect devem ser arquivos em texto puro no formato csv, separado por “,” e decimal “.”. Eles são criados em um diretório local no computador para depois serem transferidos para o ODK Collect.

Importante

Use o caminho do objeto `dirODK` para um diretório local do seu computador onde pretende salvar os arquivos de media do ODK Collect. Este vai se o diretório de onde os arquivos são transferidos do computador para o dispositivo móvel com o ODK Collect.

```
dirODK <- "/home/aao/Ale2026/AleProjetos/Rpackages/odkData"
```

Códigos das subparcelas

Os arquivos `xcode.csv` e `ycode.csv` armazenam os códigos das subparcelas do plot e tem duas colunas `label` e `name`, que identificam as subparcelas no eixo x e y, respectivamente. As colunas podem ter a mesma informação, mas ambas são necessárias.

```
xCode <- LETTERS[1:16]
```

```

yCode <- sprintf("%02d", 0:15)
write.table(data.frame(name = xCode, label = xCode), file = file.path(dirODK,"xcode.csv"),
  sep = ',', row.names = FALSE)
write.table(data.frame(name = yCode, label = yCode), file = file.path(dirODK,"ycode.csv"),
  sep = ',', row.names = FALSE)

```

Dados das árvores do último censo e códigos de subparcelas

O `lastcensus.csv` é um arquivo de dados do censo anterior. Veja o `help(lastCensusODK)`. Os dados `peicCensus2019` são distribuídos junto ao pacote em um arquivo `.rda`.

```

data(package = "odkTreeCensus")
data("lastCensusPEIC", package = "odkTreeCensus")
str(peicCensus2019)
lastCensus <- lastCensusODK(data = peicCensus2019 , splitX = 5, splitY = 5, maxX = 20, maxY
  = 20, subPlotCodeX = xCode, subPlotCodeY = yCode, dirData = dirODK, save = TRUE)
str(lastCensus)

```

Equipe

O arquivo `team.csv` contém duas colunas: `name` e `label` nome e apelido dos membros da equipe de campo.

```

lb <- c("Alexandre Adalardo", "Marcel Zimmermann", "Luisa Truffi")
nm <- c("AAO", "GIL", "LU")

write.table(data.frame(name = nm, label = lb), file = file.path(dirODK,"team.csv"), sep =
  ',', row.names = FALSE)

```

Lista de Espécies

A lista de espécies contempla todos os nomes, válidos, morfotipos e `indet` para todos os níveis taxonômicos a partir de família. O arquivo deve-se chamar `splist.csv`. A função `spList()` cria no formato adequado à partir de um arquivo `fam` e `species`. Gera `gen` se não houver.

```

str(peic19)
splist <- spList(data = lastCensus, dirData = dirODK, saveFile = TRUE)
str(splist)

```

Dados de tags por subquadrat

Essa informação é importante para a lista de tags não encontradas, o formato é de uma coluna subq como o código do subquadrat e tags contendo todas as tags do subquadrat separadas por espaço no mesmo campo.

```
subqTags(data = lastCensus, dirData = dirODK, saveFile = TRUE)
```

Mapas e Grids svg

Os mapas e grids do formulário foram feitos em svg para permitir a seleção de elementos na imagem. Para isso usamos os pacotes grid e gridSVG.

Os rótulos para os svg

Para que os elementos do svg sejam selecionados é preciso que os rótulos sejam opções na planilha choices do formulário xlsx. Quando há muitas opções é preferível incorporá-las em um arquivo csv.

Como os mapas incorporam uma borda além do subquadrat é preciso fazer um objeto de dados de mapas, completo. Esse arquivo é uma lista grande quando é feito para a parcela toda, mas pode ser feito parcialmente para subparcelas.

i Árvores vivas

O arquivo é contruído apenas com as árvores vivas no último censo, apesar de ser desejável ter árvores morta e desaparecidas, provavelmente deixaria o mapa muito poluído.

```
subplot <- sort(unique(lastCensus$quad))
mapdata <- mapData(lastCensus[lastCensus$status == "A", ], subplotCodes = subplot, splitX =
  5, splitY = 5, maxX = 20, maxY = 20, buffer = 2)
sk <- subqKey(dataMap = mapdata, dirData = dirODK, saveFile = TRUE)
str(sk)
sk[sk$subq == "A00_0x0", ]
```

gridid.csv

```
subqSize = 5
gridSize = .2
xseq <- rep(seq(0.0, subqSize - gridSize, by = gridSize), each = subqSize/gridSize)
yseq <- rep(seq(0.0, subqSize - gridSize, by = gridSize), subqSize/gridSize)

xyid <- paste("x = ",sprintf("%.1f", xseq), "; y = ", sprintf("%.1f", yseq),"; .1",
  sep="")
write.table(data.frame(xyid = xyid, xseq = xseq, yseq = yseq), file = file.path(dirODK,
  "gridid.csv"),row.names = FALSE, sep = ",")
```

```
xyid <- read.table(file = file.path(dirODK, "gridid.csv"), header = TRUE, sep = ",")
str(xyid)
```

Seleção de subquadrat

O mapa que gera a figura com os subquadrats para serem selecionados. Preciso ajustar os argumentos para ficar no padrão das outras funções.

```
selSubq(xmax = 20, ymax = 20, subqX = 5, subqY = 5, mapSize = c(10,10), fontSize = 14,
        wd2save = dirODK)
```

Mapas e Grids para seleção e mapeamento de árvores

Para produzir os mapas com as localizações das árvores, precisa criar o objeto de dados com a função `mapData`. Aqui estou criando os dados apenas para a coluna A como exemplo. Os dados de mapa, assim como os mapas e grids são arquivos grandes que não devem ser carregados todos de uma vez no dispositivo móvel. Para os grids de mapeamento, esse objeto não é necessário, já que não há buffer para o mapeamento.

```
subplotA <- sort(grep("A", unique(lastCensus$quad), value = TRUE))
mapdataA <- mapData(lastCensus[lastCensus$status == "A", ], subplotCodes = subplotA, splitX
                 = 5, splitY = 5, maxX = 20, maxY = 20, buffer = 2)
str(mapdataA)
nsuqA <- length(mapdataA)
#subplotA <- paste("A", sprintf("%02d", 0:15), sep = "")

for(i in subplotA)
{
  svgMap(mapdataA, subPlotCode = i, mapSize = c(12, 12), fontSize = 12, wd2save =
        file.path(dirODK, "subplotA"))
  svgGrid(censoData = lastCensus, subPlotCode = i, mapSize = c(12, 12), fontSize = 12,
        wd2save = file.path(dirODK, "subplotA"))
}
```

ODK Collect

Dispositivo android

O dispositivo android precisa ter a “Depuração USB” ativada, para isso é preciso desbloquear as “Opções de desenvolvedor” Conecte o dispositivo android e garanta que ele tem “Opções de desenvolvedor” e “Depuração USB” ativados. Caso não esteja, veja como [ativar opção de desenvolvedor](#).

Entre na “opções de desenvolvedor” nas configurações do android e ative “Depuração USB”.

Android Debug Bridge

O ADB é necessário para que o computador conectado ao dispositivo conecte e transfira arquivos pelo pacote.

No linux (Debian ou Ubuntu) use:

```
sudo apt update
sudo apt install android-tools-adb
```

No archlinux:

```
sudo pacman -S android-tools
```

No windows, consulte: [android debug bridge](#)

Preparando o dispositivo móvel

O [Open Data Kit](#) tem um mecanismo próprio para instalar formulários e retornar os dados coletados, chamado **ODK Central** que precisa estar instalado em um servidor para se comunicar com o **ODK Collect**. O servidor tem o **ODK Central** configurado para testes, sempre guarde uma cópia local dos arquivos originais coletados pelo dispositivo móvel, antes de apagá-los.

Criando um projeto ODK Collect

Entre no aplicativo **ODK Collect** e siga os passos solicitado para criar um novo projeto inserindo os dados diretamente no app selecionando a opção **Entrar com os detalhes do projeto manualmente** (Figura~1 a). Além de um usuário e senha, que precisam ser criados, há a necessidade de incluir uma url para a conexão com o **ODK Central**. Aqui inclua a url do **ODK Central** em nosso servido (<http://odk.ib.usp.br>). Não se preocupe com nenhum desses dados iniciais, apenas crie o projeto e preencha os campos solicitados (Figura~1 b). Caso seja solicitado, permita que o ODK Collect emita notificações no seu dispositivo móvel (Figura~1 c)

Para incluir o formulário no **ODK Collect** precisamos de um projeto criado no app e acesso à memória interna do dispositivo Android com permissão para edição das pastas do **ODK Collect**.

Conectando o Dispositivo Android

Após conectar o tablet ao computador abra as notificações e nas opções de conexão USB selecione a *Transferir arquivos/Android Auto*. Mantenha o dispositivo ativo e sem bloqueio de tela.

Configurando o USB Padrão

Para tornar o dispositivo android sempre configurado para a troca de arquivos pelo usb, conecte ao computador, vá para:

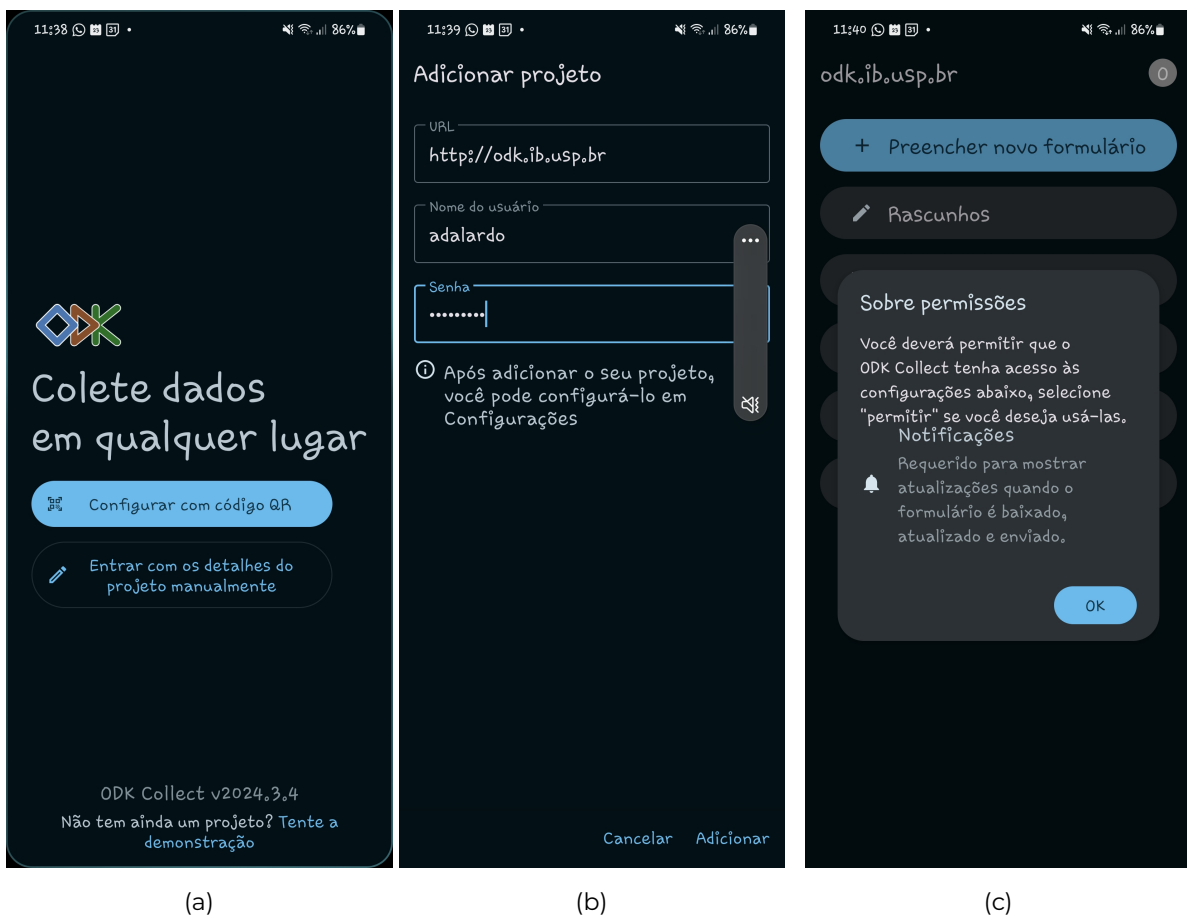


Figura 1: Criando conta manualmente no ODK Collect

- Configurações
- Opções do desenvolvedor
- Configurações USB padrão
- selecione: “Transferindo arquivos/Android Auto”
- desconecte o cabo USB e reconecte
 - confira a Configuração USB padrão
 - caso mude para “Apenas carregar o tablet”
 - selecione novamente “Transferindo arquivos/Android Auto”
 - desconecte e reconecte o cabo USB para verificar se a configuração permanece.

Criar projeto no ODK Collect

O dispositivo móvel deve ter instalado o app ODK Collect e ter um projeto iniciado. A url do projeto pode ser <https://odk.ib.usp.br>, onde há um ODK Central instalado.

Atualizar o formulário xml

Aqui só é necessário atualizar se houve alguma alteração no formulário `treeCensusForm.xlsx` depois da instalação do pacote.

O formulário é criado em xlsx no libreOffice. É preciso traduzir para formato xml utilizando o pacote do python pyxform. O pyxform foi incorporado no pacote do R, assim como o `treeCensusForm.xlsx` e o `treeCensusForm.xml`.

Atenção

Este procedimento é dispensável, o pacote já é distribuído com o formulário xml traduzido e atualizado. Ele só deve ser usado se houver alguma mudança local no xlsxform (planilha Libre-Office) que precisa ser incorporada no formulário, em casos de mudanças locais emergenciais. Caso seja um bug, deve ser informado no issues do github para a solução seja incorporada na distribuição do pacote.

```
xlsxFile <- system.file("odk/treeCensusForm.xlsx", package = "odkTreeCensus")
pkgDirForm <- dirname(xlsxFile)
xmlFile <- file.path(pkgDirForm, "treeCensusForm.xml")
xmlForm(xlsxPath = xlsxFile, xmlPath = xmlFile)
```

Copia o caminho do formulário XML

Copiando o caminho onde se encontra o formulário xml do pacote para transferir ao dispositivo móvel.

```
xmlDir <- system.file("odk", package = "odkTreeCensus")
xmlPath <- system.file("odk/treeCensusForm.xml", package = "odkTreeCensus")
```

Subindo o formulário para o dispositivo móvel android

```
pushToODKCollect(localDir = xmlDir, formName = "treeCensusForm")  
mediaToODK(localMediaDir = dirODK, formName = "treeCensusForm")
```